Software Testing Fundamentals: Verification, Validation, and Testing Levels

Software Engineering & Project Management • Student Learning Book

Learning outcomes

• Explain why software testing is needed and distinguish errors, defects, and failures.

• Differentiate verification from validation.

• Describe unit, integration, system, and acceptance testing.

• Recognize the roles of developers, testers, customers, and users in quality activities.

• Plan a simple testing approach for a College Event Management System (CEMS).

Running case study: CEMS lets students browse college events, register, receive confirmation, and lets organizers manage attendance.

1. Why Do We Test Software?

Software can fail because requirements were misunderstood, logic was incorrect, interfaces did not match, data was unexpected, or the deployed environment behaved differently from the development environment. Testing provides evidence about the quality of a product and helps reveal problems before they affect users.

Imagine that 500 students try to register for a college festival at the same time. The page may look correct in a demonstration but still fail when many students submit requests, when an event becomes full, or when the notification service is unavailable. Testing should explore these realistic conditions—not only the ideal path.

Key idea: Testing can reveal the presence of defects, but passing tests cannot prove that a system has no defects at all.

2. Error, Defect, and Failure

Term	Meaning	CEMS example
Error / mistake	A human action or misunderstanding that introduces a problem	A developer misunderstands the rule for event capacity.
Defect / bug	A flaw in a requirement, design, code, or other work product	The registration logic allows one more student than the event capacity.
Failure	Observable incorrect behavior when the software runs	The 101st student receives a confirmed registration for an event limited to 100.

A defect does not always cause a visible failure: the faulty code may not execute under the current conditions. Testing therefore needs a thoughtful range of inputs, states, and environments.

3. Verification and Validation

Question	Verification	Validation
Core question	Are we building the product right?	Are we building the right product?
Focus	Conformance to specifications, designs, standards, and work products	Whether the product satisfies user needs and intended use

Question	Verification	Validation
Examples	Review an SRS; inspect a design; check code against a requirement	Ask students to complete registration and evaluate whether the workflow meets their needs
Common activities	Reviews, inspections, static analysis, and other checks	Acceptance testing, user evaluation, and execution-based testing against needs

Verification is not limited to running code. A requirements review or design inspection can find issues before the application is executable. Validation often involves exercising the product or evaluating it with representative users, although the exact method depends on the product and lifecycle.

> **Example:** The SRS says an event page must show the registration deadline. Checking the design and implementation against that statement supports verification. Asking students whether the displayed deadline and registration process meet their actual needs supports validation.

4. Testing Levels

4.1 Unit testing

Unit testing checks a small, isolated unit such as a function, method, or class. Developers commonly write these tests, often using test frameworks and substitutes for external dependencies.

CEMS example: Test `hasCapacity(registeredCount, capacity)` with 99/100, 100/100, and 101/100 inputs. Expected results should reflect the agreed business rule.

4.2 Integration testing

Integration testing checks whether connected modules or services work together correctly.

CEMS example: Check that the registration service saves a registration through the data-access layer and receives the correct event-capacity information. Test API-to-database interaction and error handling.

4.3 System testing

System testing evaluates the complete integrated application against its requirements. It can include functional behavior and non-functional qualities such as performance, security, accessibility, and reliability.

CEMS example: Start from login, browse events, register, view confirmation, and verify that the registration appears in the organizer dashboard.

4.4 Acceptance testing

Acceptance testing evaluates whether the system is suitable for its intended users and business purpose. It may be conducted by customers, representatives, or end users using agreed acceptance criteria.

CEMS example: College event coordinators verify that they can create an event, set a capacity, view the attendee list, and record attendance before approving release.

Level	Typical scope	Example
Unit	Single function/class	Capacity rule
Integration	Connected components	Registration service + database
System	Complete application	End-to-end event registration
Acceptance	User/business needs	Coordinator approves event workflow

5. How the Levels Fit Together

A team usually benefits from checking small pieces early and then progressively checking broader interactions. The exact sequence and ownership vary by development approach; these levels are not restricted to one specific SDLC model.

```
Unit tests: Does this piece behave correctly?
↓
Integration tests: Do connected pieces work together?
↓
System tests: Does the integrated product satisfy its requirements?
↓
Acceptance tests: Is it acceptable for intended use?
```

Finding a problem early can reduce the cost of diagnosing it because fewer components and assumptions are involved. However, unit tests alone cannot establish that the complete system meets user needs.

6. Functional and Non-Functional Testing Examples

Area	Example test	Expected result
Functional	Register while registration is open and capacity remains	Registration is saved and confirmation is shown
Functional	Register when event is full	Registration is rejected with a clear message
Functional	Submit the same registration twice	Duplicate is prevented according to the business rule
Security	A student attempts to edit event capacity	Access is denied
Performance	Many users open the event list at once	Response stays within the agreed performance target
Reliability	Notification provider is unavailable	Registration data remains correct and notification failure is handled safely
Usability	Student uses the form on a phone	Labels and errors are clear and the form is usable

A good test case specifies the preconditions, inputs or actions, and expected result. Performance and security targets should be agreed with stakeholders rather than invented by the tester.

7. Static and Dynamic Testing

Static testing

Static testing examines work products without executing the software. Examples include reviewing requirements, inspecting UML diagrams, peer code review, and using static-analysis tools.

Dynamic testing

Dynamic testing executes software with selected inputs and compares actual behavior with expected behavior. Unit, integration, system, and acceptance tests can involve dynamic execution.

> These are different ways to classify testing: testing levels describe scope, while static/dynamic describes whether execution occurs. For example, a code review is static; a unit test that calls a function is dynamic.

8. Test Case Design: A Simple Example

Requirement: “A student can register for an event only if registration is open and capacity remains.”

ID	Precondition / input	Expected result
TC-01	Registration open; 20 of 100 places used	Registration succeeds
TC-02	Registration closed; capacity remains	Registration is rejected
TC-03	Registration open; 100 of 100 places used	Registration is rejected as full

ID	Precondition / input	Expected result
TC-04	Registration open; same student already registered	Duplicate is prevented
TC-05	Database write fails during registration	No false success is shown; data remains consistent

Before execution, define test data and expected results. During execution, record actual results. If actual and expected results differ, capture evidence and report a defect with reproducible steps.

9. Defect Reporting

A useful bug report helps another person reproduce and diagnose the problem. It commonly contains:

• Short title and unique ID

• Environment and build/version

• Preconditions and test data

• Steps to reproduce

• Expected result and actual result

• Severity/impact and priority for fixing

• Screenshot, log, or other evidence

• Status and retest result

Example title: “Registration succeeds when event capacity is already full.” Describe the event setup, exact steps, expected rejection, actual confirmation, and supporting evidence.

10. Classroom Activity: Test CEMS

In pairs, write six test cases for event registration: two successful paths, three failure/edge cases, and one non-functional test. Exchange test cases with another pair and check whether the expected results are precise.

Deliverable: a small test-case table with IDs, preconditions, steps/inputs, expected results, actual results, and pass/fail status.

11. Review Questions with Suggested Answers

1. Why is testing necessary?

Suggested answer: It provides evidence about quality and helps reveal defects before or after release.

2. Distinguish an error, a defect, and a failure.

Suggested answer: An error is a human mistake; a defect is a flaw in a work product; a failure is observable incorrect behavior during execution.

3. What question does verification ask?

Suggested answer: Are we building the product right according to specifications and standards?

4. What question does validation ask?

Suggested answer: Are we building the right product for users and intended use?

5. Give a unit-testing example.

Suggested answer: Test the event-capacity function with available and full-capacity values.

6. What is integration testing?

Suggested answer: Checking whether connected modules or services interact correctly.

7. How does system testing differ from acceptance testing?

Suggested answer: System testing evaluates the integrated product against requirements; acceptance testing evaluates suitability against user or business acceptance criteria.

8. Give one example of static testing.

Suggested answer: Reviewing an SRS or inspecting source code without executing it.

9. What should a bug report include?

Suggested answer: Reproduction steps, expected and actual results, environment, impact, and evidence.

10. Can passing tests prove that no defects remain?

Suggested answer: No. Tests reveal defects when exercised conditions expose them; they cannot exhaustively prove defect absence.

12. Glossary

Term	Meaning
Test case	Defined conditions, inputs/actions, and expected result used to check behavior
Verification	Checking conformance to specified requirements or work products
Validation	Checking suitability for user needs and intended use
Unit test	Test of a small isolated unit
Integration test	Test of interactions among components
System test	Test of the integrated application as a whole
Acceptance test	Evaluation against user or business acceptance criteria
Static testing	Review or analysis without executing the software
Dynamic testing	Testing by executing the software
Regression testing	Retesting to check that changes have not broken existing behavior
Defect	A flaw in a requirement, design, code, or other work product

13. Homework

1. Write eight test cases for a library book-borrowing system, including unavailable books and invalid user status.
2. For three sample defects, distinguish the human error, the underlying defect, and the observed failure.
3. Create a simple defect report for one test that fails.
4. Explain why a project needs unit, integration, system, and acceptance testing rather than only one type.
5. Find one example where a requirements review could detect a problem before coding begins.

Next chapter: Black-box and white-box testing, test design techniques, and regression testing.